การบ้าน Coordination with ZooKeeper

บทนำ

การบ้านนี้เกี่ยวข้องกับ coordination โดยใช้ ZooKeeper เพื่อทำ distributed lock ให้ process หลายตัวผลัดกันเข้า critical section ได้ทีละตัว ตามวิธีที่อธิบายในเลคเชอร์

ทำเสร็จแล้วนิสิตควรจะ:

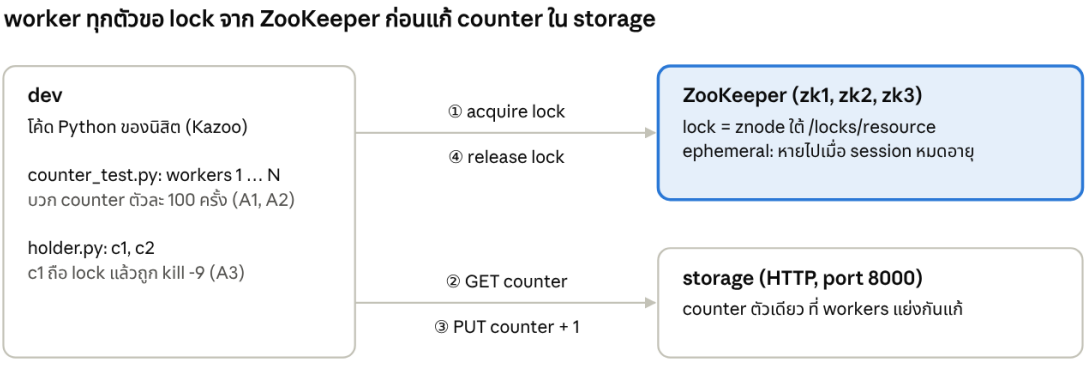
- เห็น lost update เมื่อหลาย process อ่าน บวก แล้วเขียนค่าเดียวกันพร้อมกันโดยไม่มี lock
- อธิบาย herd effect ของ Lock v1 จากตัวเลขที่วัดได้
- เขียน Lock v2 ด้วย ephemeral sequential znode และ watch แล้วอธิบายได้ว่าทำไมจึงแก้ herd effect ได้
- อธิบายได้ว่า session timeout และ ephemeral znode ทำให้ lock ถูกปล่อยเมื่อผู้ถือ crash ได้อย่างไร

กติกา

- ทำเป็นกลุ่ม กลุ่มละ 2-3 คน
- ใช้ AI ช่วยเขียนโค้ดได้ แต่ผลการทดลองต้องมาจากการรันโปรแกรมของนิสิตเอง และนิสิตต้องเข้าใจโค้ดที่ส่ง
- คะแนนส่วนใหญ่อยู่ที่คำอธิบาย ว่าเห็นอะไร และ**ทำไม**จึงเป็นเช่นนั้น
- ข้อสอบปลายภาคอาจมีคำถามที่เกี่ยวข้องกับการบ้านนี้

การเตรียมเครื่อง (Set-up)

ทุกอย่างรันบน Docker จากชุดไฟล์ `hw1_coordination.zip` ซึ่งมี 3 อย่าง: ZooKeeper 3 servers (`zk1`–`zk3`) ที่เก็บ lock, storage service ที่เก็บ counter ตัวเดียว และ container ชื่อ `dev` ซึ่งเป็นที่ที่นิสิตรันโค้ด Python ทั้งหมด



dev, ZooKeeper และ storage

ทุก worker ทำตามลำดับ ①–④ ถ้าไม่มีขั้น ① และ ④ worker หลายตัวจะทำ ②–③ ซ้อนกันได้

สิ่งที่ต้องมี: Docker Desktop (Windows/macOS) หรือ Docker Engine ที่มี Compose v2 (Linux) ตั้ง memory ให้ Docker อย่างน้อย 2 GB

เริ่มระบบ แตก `hw1_coordination.zip` แล้วรันบน host ในโฟลเดอร์ `hw1_coordination`:

```
docker compose up -d            # start 3 ZooKeeper servers, storage and dev; wait ~20 s
docker compose exec dev bash    # a shell in dev
```

คำสั่งที่สองเปิด shell ใน container `dev` คำสั่ง `python` ทุกคำสั่งในการบ้านนี้รันใน shell นี้ (ข้อ A3 ต้องใช้ 3 shells ให้รันคำสั่งที่สองบน host 3 ครั้ง) จากนั้นตรวจว่ามี leader แล้ว:

```
python scripts/zkstat.py          # inside dev: which server is the leader?
```

ถ้ายังไม่มี server ไหนเป็น `leader` ให้รอสักครู่แล้วรันใหม่

เมื่อเลิกใช้ ให้รัน `docker compose down -v` บน host เพื่อหยุดและลบข้อมูลทั้งหมด

ไฟล์ที่ใช้ในการบ้านนี้

ทุกไฟล์ที่นิสิตต้องรัน อ่าน หรือแก้ อยู่ในโฟลเดอร์ `scripts/` ส่วนโฟลเดอร์ `lib/` เป็นโค้ดเสริมที่ scripts เรียกใช้ (เช่น `lib/zkutil.py` และ storage service) นิสิตไม่ต้องแก้

ไฟล์	ใช้ทำอะไร
`scripts/counter_test.py`	ให้มาแล้ว: รัน workers หลายตัวที่บวก counter ตัวเดียวกัน (A1, A2)
`scripts/lock.py`	`NoLock` และ `LockV1` ให้มาแล้ว **`LockV2` นิสิตต้องเขียนเอง** (ดู `TODO`, ข้อ A2)
`scripts/holder.py`	ให้มาแล้ว: ขอ lock แล้วถือไว้ ใช้ทดลอง crash (A3)

ปัญหาเรื่อง setup ดูวิธีแก้ได้ใน `README.md`

โจทย์ (Instructions)

ทำตามลำดับ A1 → A2 → A3 เพราะแต่ละข้อใช้ผลของข้อก่อนหน้า นิสิตจะได้ดูการทำงานของ Lock v1 จากเลคเชอร์เรื่อง ZooKeeper ก่อน เพื่อให้เห็น herd effect จากนั้นจึงเขียน Lock v2 ที่แก้ปัญหานั้น และสุดท้ายดูว่าเกิดอะไรขึ้นเมื่อผู้ถือ lock เกิดการ crash

A1. ไม่มี lock เทียบกับ Lock v1

`scripts/counter_test.py` สร้าง worker process หลายตัว แต่ละตัวบวก 1 เข้าไปใน counter ตัวเดียวกัน 100 ครั้ง ด้วยการอ่านค่าปัจจุบัน → บวกเพิ่มเข้าไป 1 → เขียนกลับ (ทำ read-modify-write) ผ่าน storage service ถ้า worker ใช้ lock ก็จะถือ lock ไว้ตลอดช่วงนั้น

ขั้นที่ 1: ไม่ใช้ lock

```
python scripts/counter_test.py --lock none
```

สังเกตค่า `counter` ที่ได้เทียบกับค่า `expected` และลองรันคำสั่งเดิมซ้ำๆ เพื่อสังเกตว่าได้ค่า counter เหมือนกันหรือไม่

ขั้นที่ 2: ใช้ Lock v1 ซึ่งให้มาแล้วใน `scripts/lock.py` ตาม recipe ในสไลด์ "Lock v1: Simple Lock"

```
python scripts/counter_test.py --lock v1 --workers 2
python scripts/counter_test.py --lock v1 --workers 5
python scripts/counter_test.py --lock v1 --workers 10
```

สังเกต `counter` และ `wakeups per acquisition` คือจำนวน watch notification ที่ workers ได้รับเฉลี่ยต่อการได้ lock หนึ่งครั้ง

ตอบในรายงาน:

1. ทำไม `--lock none` จึงได้ counter น้อยกว่าที่ควรเป็น ยกตัวอย่างลำดับเหตุการณ์ของ worker 2 ตัวที่ทำให้ update หายไป 1 ครั้ง
2. Lock v1 ได้ counter ถูกต้องไหม `wakeups per acquisition` เปลี่ยนอย่างไรเมื่อจำนวน workers เพิ่มขึ้น เพราะอะไร และปัญหานี้เรียกว่าอะไร
3. เปิดอ่าน `LockV1.acquire()` ใน `scripts/lock.py` บรรทัดไหนป้องกัน race condition ที่สไลด์ Lock v1 พูดถึง และป้องกันได้อย่างไร

A2. เขียน Lock v2 แล้วเปรียบเทียบ

เขียน `acquire()` และ `release()` ของ `LockV2` ใน `scripts/lock.py` ตามขั้นตอนในสไลด์ "Lock v2: Sequential Ephemeral Znodes" (ขั้นตอนเดียวกันอยู่ใน docstring ของ class ด้วย) ห้ามใช้ `kazoo.recipe` และให้นับ watch notification ทุกครั้งที่ได้รับใน `self.wakeups` (ดูตัวอย่างใน `LockV1`)

ทดสอบด้วยจำนวน workers เท่ากับข้อ A1:

```
python scripts/counter_test.py --lock v2 --workers 2
python scripts/counter_test.py --lock v2 --workers 5
python scripts/counter_test.py --lock v2 --workers 10
```

ทำตารางเปรียบเทียบ v1 กับ v2: workers, counter, wakeups per acquisition, elapsed

ตอบในรายงาน:

1. อธิบายการทำงานของ `acquire()` ที่เขียน ใน 3–5 ประโยค
2. ทำไม v2 จึงได้ wakeups per acquisition ประมาณ 1 ไม่ว่าจะมีกี่ workers และทำไม elapsed ของ v2 จึงต่างจาก v1
3. znode ตัวก่อนหน้าหายไป แต่ znode ของเรายังไม่ใช่ตัวที่เลขน้อยที่สุด เกิดขึ้นได้อย่างไร และโค้ดของนิสิตจัดการอย่างไร

A3. เมื่อผู้ถือ lock crash

`scripts/holder.py` ขอ lock แล้วถือไว้ 60 วินาที พร้อมพิมพ์ PID (process ID) และเวลาของทุกขั้นตอน

โปรแกรมนี้ใช้ Lock v2 จากข้อ A2 (ถ้ายังทำ A2 ไม่เสร็จ เติม `--lock v1` ท้ายคำสั่งได้)

ขั้นที่ 1:

ให้นิสิตเปิด shell ใน dev 3 อัน (โดยรัน `docker compose exec dev bash` บน host 3 ครั้ง ใน 3 terminal) แล้วรันคำสั่งต่อไปนี้บน terminal ที่ระบุ

```
python scripts/holder.py --name c1    # terminal 1: c1 gets the lock
python scripts/holder.py --name c2    # terminal 2: start it after c1 prints ACQUIRED
```

(ก) สังเกตผลว่าเมื่อ c1 hold lock ไว้ครบ 60 วินาทีแล้ว มีการเปลี่ยน lock holder หรือไม่ และ c2 ได้ lock เร็วแค่ไหนหลังจาก c1 ปล่อย lock

(ข) หลังจาก c1 จบแล้ว c2 จะเป็น lock holder ให้รัน c1 ใหม่ใน terminal 1 (คราวนี้ c1 จะเป็นฝ่ายรอ lock) แล้ว kill c2 ใน terminal 3 ก่อนที่ c2 จะถือ lock ครบ 60 วินาที (ดู PID ของ c2 จากบรรทัดแรกที่ c2 พิมพ์ pid=...) สังเกตผลว่ามีการเปลี่ยน lock holder หรือไม่ และ c1 ได้ lock หลังจาก kill นานเท่าไร

```
python scripts/holder.py --name c1    # terminal 1: c1 now waits for the lock
kill -9 <PID of c2>                   # terminal 3: crash the lock holder
```

ขั้นที่ 2:

ใน `LockV2.acquire()` ของ `scripts/lock.py` ให้เปลี่ยน `ephemeral=True` เป็น `ephemeral=False` (ถ้าใช้ `--lock v1` ให้แก้ใน `LockV1.acquire()`)

หยุดโปรแกรม holder.py ที่รันอยู่ทั้งหมด (Ctrl-C) แล้วรัน c1 และ c2 ใหม่เหมือนในขั้นที่ 1 จากนั้น kill c1 ซึ่งเป็น lock holder ขณะที่ c2 รออยู่ สังเกตผลว่าเมื่อ kill lock holder แล้วเกิดอะไรขึ้น

```
python scripts/holder.py --name c1    # terminal 1: c1 gets the lock
python scripts/holder.py --name c2    # terminal 2: start it after c1 prints ACQUIRED
kill -9 <PID of c1>                   # terminal 3: crash the lock holder
```

หลังการทดลองให้แก้กลับคืนด้วย lock.py ที่ส่งต้องเป็น `ephemeral=True`

(หมายเหตุ เมื่อเปลี่ยนเป็น persistent node แล้ว lock จะค้างอยู่ แม้ว่าจะแก้ lock.py ให้ `ephemeral=True` แล้วก็ตาม ถ้าหากต้องการลบ lock znode ที่ค้างอยู่ต้องรัน `docker compose down -v` และ `docker compose up -d` บน host อีกรอบ)

ตอบในรายงาน:

1. ในขั้นที่ 1 (ข) เมื่อ lock holder ถูก kill มีอะไรเกิดขึ้นบ้าง c1 ได้ lock หลังจาก kill นานเท่าไร และทำไมจึงช้ากว่าตอนที่ c1 ปล่อย lock ตามปกติใน (ก)
2. ในขั้นที่ 2 ถ้า lock znode เป็น persistent แทนที่จะเป็น ephemeral จะได้ผลอย่างไร เพราะเหตุใด

การส่งงาน

ส่งภายในวันเวลาที่ระบุใน MyCourseView เป็น zip ไฟล์เดียว ตั้งชื่อ `hw1_<รหัสนิสิตทุกคนในกลุ่ม คั่นด้วย _>.zip` ซึ่งประกอบด้วย:

- `report_hw1.pdf`
  - ชื่อ-นามสกุล และรหัสนิสิต ของสมาชิกในกลุ่ม
  - แต่ละข้อประกอบด้วย ภาพ capture หน้าจอที่รันคำสั่ง และคำถาม-คำตอบ
- `scripts/lock.py` ที่เขียน `LockV2` แล้ว เมื่อ copy ทับชุดไฟล์เดิมต้องรันได้